

Department of Information Technology

SQL Lab

Experiment : 02

Experiment No.	Experiment No. 2	Group No	1
Subject	SQL Lab	Class/Branch	SE/IT
Title	Relational Schema Mapping	Roll nos	7,10,17

1. Experiment Number

Experiment No. 2

2. Aim

To map the Entity-Relationship (ER) / Extended Entity-Relationship (EER) model developed in Experiment No. 1 (Hospital and Healthcare Management System) into an appropriate relational schema by applying standard ER-to-Relational mapping rules.

3. ER / EER Diagram

Below is the architectural representation of the Entity-Relationship Diagram created for the Hospital and Healthcare Management System (Experiment No. 1).

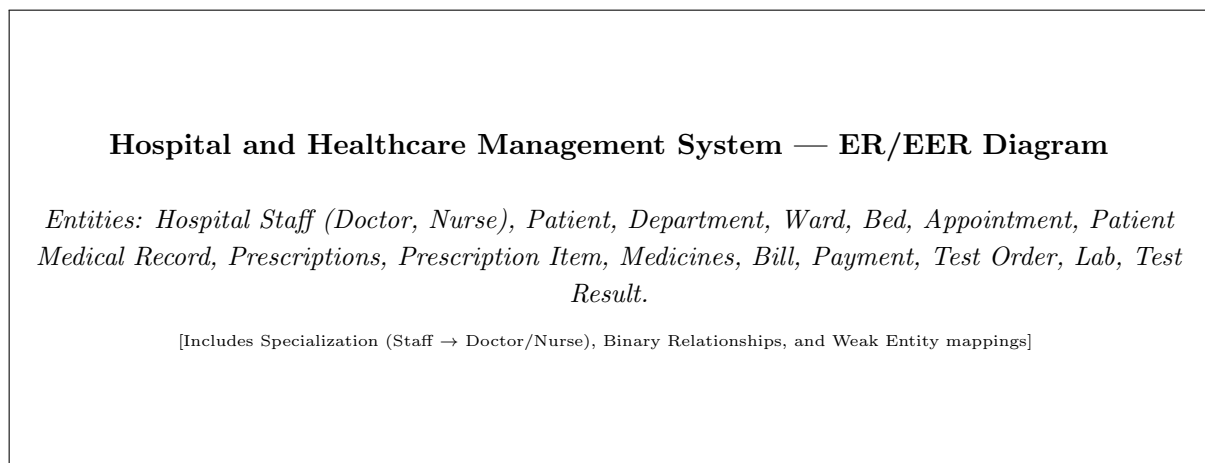


Figure 1: ER/EER Model for Hospital and Healthcare Management System

4. Identification of Entities, Attributes and Keys

The entities, their respective attributes, attribute classifications, and key attributes identified from the ER diagram are summarized below:

Entity Name	Attributes	Attribute Type	Primary Key / Key
Hospital Staff	Staff_ID, First Name, Middle Name, Last Name, Gender, Street Name, City, State, Zip-code, Blood Group, Contact	Composite (Name, Address), Simple	Staff_ID
Doctor	Doctor_ID, Department_ID, Experience	Simple	Doctor_ID (Inherits Staff_ID)
Nurse	Nurse_ID, Age, Shift Timing	Simple	Nurse_ID (Inherits Staff_ID)
Patient	Patient_ID, First Name, Middle Name, Last Name, Gender, D.O.B, Street Name, City, State, Zipcode, Blood Group, Contact	Composite (Name, Address), Simple	Patient_ID
Department	Department_ID, Department Name, Department Location, Description	Simple	Department_ID
Ward	Ward_ID, Ward Type, Location, Capacity	Simple	Ward_ID
Bed	Bed_ID, Ward_ID, Bed Type, Status	Weak Entity Attribute	(Ward_ID, Bed_ID)
Appointment	Appointment_ID, Patient_ID, Doctor_ID, Appointment Date, Time Slot, Status	Simple, Foreign Keys	Appointment_ID
Patient Medical Record	Record_ID, Patient_ID, Doctor_ID, Diagnosis, Treatment, Symptoms, Advice	Simple, Foreign Keys	Record_ID
Prescriptions	Prescription_ID, Record_ID, Prescription Date, Instructions	Simple, Foreign Key	Prescription_ID
Medicines	Medicine_ID, Medicine Name	Simple	Medicine_ID
Prescription Item	Prescription_ID, Medicine_ID, Dosage, Duration, Frequency	Relationship Attributes	(Prescription_ID, Medicine_ID)
Bill	Bill_ID, Patient_ID, Bill Date, Total Amount, Discount, Final Amount, Status	Derived (Final Amount), Foreign Key	Bill_ID
Payment	Payment_ID, Bill_ID, Amount, Payment Mode, Payment Date	Simple, Foreign Key	Payment_ID
Test Order	Test_No, Patient_ID, Doctor_ID, Appointment_ID, Test Name, Order Date, Status, Lab_ID	Simple, Foreign Keys	Test_No
Lab	Lab_ID, Lab Name, Location, Contact	Simple	Lab_ID
Test Result	Result_ID, Test_No, Result Date, Result Report	Simple, Foreign Key	Result_ID

5. Mapping of Strong Entities

Applying **Rule 3.1**, one relation is created for each strong entity. The simple attributes become columns and the key attribute becomes the primary key.

1. **DEPARTMENT** (Department_ID, Department_Name, Department_Location, Description)
2. **WARD** (Ward_ID, Ward_Type, Location, Capacity)
3. **MEDICINES** (Medicine_ID, Medicine_Name)
4. **LAB** (Lab_ID, Lab_Name, Location, Contact)

6. Mapping of Attributes

6.1 Composite Attributes (Rule 3.2)

Composite attributes are flattened into simple atomic component attributes within the relation:

- **Name** → First_Name, Middle_Name, Last_Name
- **Address** → Street_Name, City, State, Zipcode

Flattened Strong Entities:

1. **PATIENT** (Patient_ID, First_Name, Middle_Name, Last_Name, Gender, DOB, Street_Name, City, State, Zipcode, Blood_Group, Contact)
2. **HOSPITAL_STAFF** (Staff_ID, First_Name, Middle_Name, Last_Name, Gender, Street_Name, City, State, Zipcode, Blood_Group, Contact)

6.2 Derived Attributes (Rule 3.4)

Attributes such as **Final_Amount** in the **Bill** entity can be calculated dynamically as (Total_Amount – Discount). Per Rule 3.4, derived attributes are omitted from physical persistent storage to prevent data inconsistency, or retained if required by application performance. Here, **Final_Amount** is represented as a calculated column.

7. Mapping Specialization / Generalization (EER Construct)

Applying **Rule 3.9**, the ER model presents a Generalization/Specialization hierarchy:

Hospital Staff (Superclass) → {Doctor, Nurse} (Subclasses)

Mapping Strategy Choice:

- **Constraint Analysis:** Total specialization with Disjoint subclasses (a staff member is either a Doctor or a Nurse, but not both).
- **Selected Option:** *Multiple relations — Superclass and Subclasses strategy.*
- **Implementation:**

- Create relation **HOSPITAL_STAFF** with primary key Staff_ID.
- Create relation **DOCTOR** with primary key Doctor_ID where Doctor_ID references HOSPITAL_STAFF (Staff_ID).
- Create relation **NURSE** with primary key Nurse_ID where Nurse_ID references HOSPITAL_STAFF (Staff_ID).

8. Mapping of Relationships (1:1, 1:N, N:1, M:N)

8.1 Binary 1:N / N:1 Relationships (Rule 3.6)

For 1:N relationships, the primary key of the 1-side is added as a foreign key on the *N*-side:

1. **Doctor Works In Department (1:N):** Place Department_ID as Foreign Key in **DOCTOR**.
2. **Nurse Assigned To Ward (1:N):** Place Ward_ID as Foreign Key in **NURSE**.
3. **Patient Books Appointment (1:N) & Doctor Handles Appointment (1:N): APPOINTMENT** (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date, Time_Slot, Status)
4. **Patient Medical Record (1:N for Patient & Doctor): PATIENT_MEDICAL_RECORD** (Record_ID, Patient_ID, Doctor_ID, Diagnosis, Treatment, Symptoms, Advice)
5. **Prescriptions Belongs To Patient Medical Record (1:N): PRESCRIPTIONS** (Prescription_ID, Record_ID, Prescription_Date, Instructions)
6. **Bill Belongs To Patient (1:N): BILL** (Bill_ID, Patient_ID, Bill_Date, Total_Amount, Discount, Final_Amount, Status)
7. **Payment Belongs To Bill (1:N): PAYMENT** (Payment_ID, Bill_ID, Amount, Payment_Mode, Payment_Date)
8. **Test Order Processed In Lab (1:N): TEST_ORDER** (Test_No, Patient_ID, Doctor_ID, Appointment_ID, Lab_ID, Test_Name, Order_Date, Status)
9. **Test Result Belongs To Test Order (1:1 / 1:N): TEST_RESULT** (Result_ID, Test_No, Result_Date, Result_Report)

8.2 Binary M:N Relationships (Rule 3.7)

The relationship between **Prescriptions** and **Medicines** is Many-to-Many ($M : N$) via the *Handles* / *Contains* relationship.

- Create a separate relation **PRESCRIPTION_ITEM**.
- Primary Key: Composite (Prescription_ID, Medicine_ID).
- Relationship attributes: Dosage, Duration, Frequency.

9. Mapping of Weak Entities

Applying **Rule 3.8**:

- **BED** is a weak entity dependent on the strong owner entity **WARD**.
- **Partial Key**: Bed_ID.
- **Owner Primary Key**: Ward_ID.
- **Primary Key of Weak Relation**: Composite (Ward_ID, Bed_ID).

BED (Ward_ID, Bed_ID, Bed_Type, Status)

10. Final Relational Schema

The complete relational schema with primary keys (**underlined**) and foreign keys (*italicized / labeled*) is presented below:

1. **HOSPITAL_STAFF** (Staff_ID, First_Name, Middle_Name, Last_Name, Gender, Street_Name, City, State, Zipcode, Blood_Group, Contact)
2. **DOCTOR** (Doctor_ID, *Department_ID*, Experience)
Foreign Key: Doctor_ID → HOSPITAL_STAFF(Staff_ID)
Foreign Key: Department_ID → DEPARTMENT(Department_ID)
3. **NURSE** (Nurse_ID, Age, Shift_Timing, *Ward_ID*)
Foreign Key: Nurse_ID → HOSPITAL_STAFF(Staff_ID)
Foreign Key: Ward_ID → WARD(Ward_ID)
4. **PATIENT** (Patient_ID, First_Name, Middle_Name, Last_Name, Gender, DOB, Street_Name, City, State, Zipcode, Blood_Group, Contact)
5. **DEPARTMENT** (Department_ID, Department_Name, Department_Location, Description)
6. **WARD** (Ward_ID, Ward_Type, Location, Capacity)
7. **BED** (Ward_ID, Bed_ID, Bed_Type, Status)
Foreign Key: Ward_ID → WARD(Ward_ID)
8. **APPOINTMENT** (Appointment_ID, *Patient_ID*, *Doctor_ID*, Appointment_Date, Time_Slot, Status)
Foreign Key: Patient_ID → PATIENT(Patient_ID)
Foreign Key: Doctor_ID → DOCTOR(Doctor_ID)
9. **PATIENT_MEDICAL_RECORD** (Record_ID, *Patient_ID*, *Doctor_ID*, Diagnosis, Treatment, Symptoms, Advice)
Foreign Key: Patient_ID → PATIENT(Patient_ID)
Foreign Key: Doctor_ID → DOCTOR(Doctor_ID)
10. **PRESCRIPTIONS** (Prescription_ID, *Record_ID*, Prescription_Date, Instructions)
Foreign Key: Record_ID → PATIENT_MEDICAL_RECORD(Record_ID)
11. **MEDICINES** (Medicine_ID, Medicine_Name)
12. **PRESCRIPTION_ITEM** (*Prescription_ID*, Medicine_ID, Dosage, Duration, Frequency)
Foreign Key: Prescription_ID → PRESCRIPTIONS(Prescription_ID)
Foreign Key: Medicine_ID → MEDICINES(Medicine_ID)

13. **BILL** (Bill_ID, *Patient_ID*, Bill_Date, Total_Amount, Discount, Final_Amount, Status)
Foreign Key: Patient_ID → PATIENT(Patient_ID)
14. **PAYMENT** (Payment_ID, *Bill_ID*, Amount, Payment_Mode, Payment_Date)
Foreign Key: Bill_ID → BILL(Bill_ID)
15. **TEST_ORDER** (Test_No, *Patient_ID*, *Doctor_ID*, *Appointment_ID*, *Lab_ID*, Test_Name, Order_Date, Status)
Foreign Key: Patient_ID → PATIENT(Patient_ID)
Foreign Key: Doctor_ID → DOCTOR(Doctor_ID)
Foreign Key: Appointment_ID → APPOINTMENT(Appointment_ID)
Foreign Key: Lab_ID → LAB(Lab_ID)
16. **LAB** (Lab_ID, Lab_Name, Location, Contact)
17. **TEST_RESULT** (Result_ID, *Test_No*, Result_Date, Result_Report)
Foreign Key: Test_No → TEST_ORDER(Test_No)

11. Conclusion

In this experiment, the Entity-Relationship (ER) and Extended Entity-Relationship (EER) diagrams for the Hospital and Healthcare Management System were successfully mapped into a complete, normalized Relational Schema. All entity sets (strong and weak), composite attributes, specialization constructs (Doctor and Nurse inheriting from Staff), binary relationships (1:1, 1:N, M:N), and referential integrity constraints (Primary and Foreign Keys) were systematically mapped according to standard relational database design rules. The generated relational schema ensures data integrity, avoids redundancy, and forms a solid foundation for physical SQL database implementation.